

Drone Choreography Using Crazyflie Quadcopters

Chiara Puggelli

Lab Experience 2025/2026

1 Introduction

This project focuses on the development of a synchronized choreography involving six Crazyflie quadcopters equipped with Lighthouse positioning decks. The system enables precise indoor localization and allows multiple drones to perform coordinated flight trajectories in a shared environment.

The main objective is to design smooth and synchronized motion patterns that allow the drones to execute a choreography reliably. This is achieved by generating trajectories based on predefined checkpoints and deploying them to each drone through the Crazyradio communication interface.

2 System Requirements

The following hardware and environmental requirements are necessary to execute the choreography:

- Six Crazyflie drones manufactured by Bitcraze
- Lighthouse positioning decks mounted on each drone
- Crazyradio PA communication module
- Indoor flight space of at least $2m \times 2m \times 2m$

The Lighthouse system provides accurate positioning, which is essential for maintaining stable formations and avoiding collisions during flight.

3 Trajectory Formation

3.1 Checkpoint Definition

The first step in the trajectory generation process consists of defining time–position pairs that represent key checkpoints in the drone motion. These checkpoints describe the structure of the choreography and determine the path followed by each drone.

The checkpoints are generated using the script:

```
python3 checkpoint.py
```

3.2 Polynomial Trajectory Generation

Smooth trajectories between checkpoints are obtained by fitting eighth-order polynomials to the time–position pairs. This is performed using the trajectory generation framework available at:

https://github.com/whoenig/uav_trajectories

Polynomial interpolation allows continuous position, velocity, and acceleration profiles, which are necessary for stable quadcopter flight.

To maintain high fidelity to the checkpoints, each trajectory segment is approximately three seconds long. The following commands generate the trajectory segments for each drone:

```
python3 scripts/generate_trajectory.py timed_waypoints_circle1.csv
part1_.csv --pieces 38
python3 scripts/generate_trajectory.py timed_waypoints_circle2.csv
part2_.csv --pieces 38
python3 scripts/generate_trajectory.py timed_waypoints_circle3.csv
part3_.csv --pieces 38
python3 scripts/generate_trajectory.py timed_waypoints_circle4.csv
part4_.csv --pieces 38
python3 scripts/generate_trajectory.py timed_waypoints_circle5.csv
part5_.csv --pieces 38
python3 scripts/generate_trajectory.py timed_waypoints_circle6.csv
part6_.csv --pieces 38
```

3.3 Trajectory Visualization

Before deploying the trajectories to the drones, it is useful to validate them through visualization. A script called `trajectory_video.py` was developed to visualize the generated uncompressed Poly4D curves.

This allows verification of trajectory smoothness and ensures that the drones maintain safe distances during the choreography.

4 Crazyflie Deployment

Each drone is assigned a specific `.csv` file containing the trajectory information. Because the onboard memory of the Crazyflie is limited, the trajectory upload is divided into two segments. The second segment is uploaded as soon as the first segment finishes executing.

All drones must operate on the same communication channel. To ensure optimal performance in indoor environments, a data rate of 2 MHz is recommended.

Before executing the choreography, all drones must be calibrated and placed close to their starting positions. Once the system is ready, the choreography can be executed with the following command:

```
python3 coreo.py
```

The program can be interrupted at any time using a keyboard interrupt if necessary.

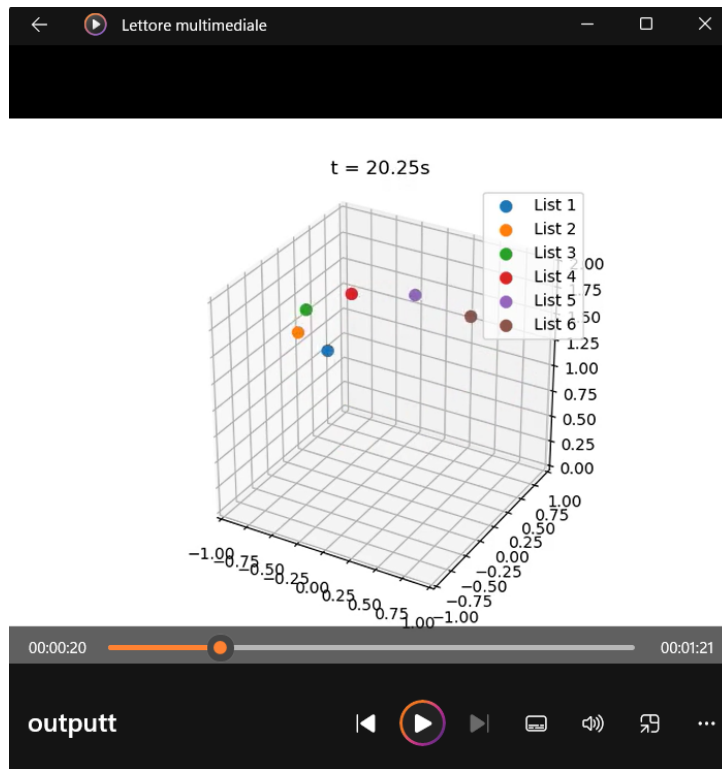


Figure 1: Visualization of the generated drone trajectories.



Figure 2: Four drones executing the choreography.